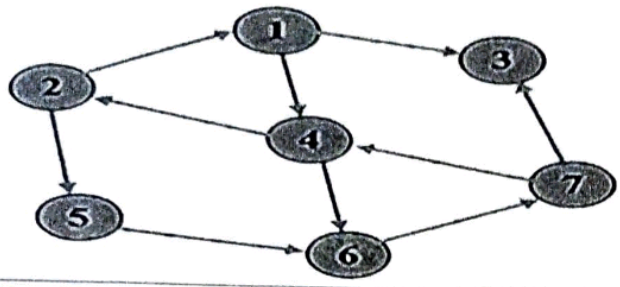
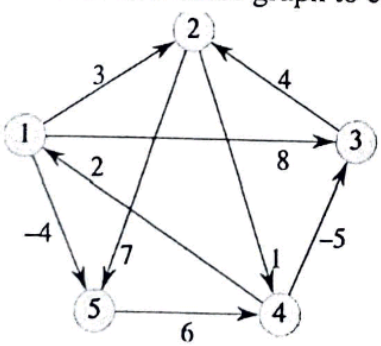
National Institute of Technology Kurukshetra
Department of Computer Engineering
End Semester Examination, Odd Semester 2025-26

Subject: Design and Analysis of Algorithms
Branch: B.Tech. M&C/AI-DS **Semester:** III
Date: 27/11/2025 **Time:** 3 Hours

Subject Code: CSPC201/DSPC203
Maximum Marks: 50
Roll No: _____

Instructions: There are 10 questions in total, and all are compulsory. Please read each question carefully before answering. Do not write programming-language–specific code, write only the algorithm where it is required.

1	(a) Solve $T(n) = \sqrt{n}T(\sqrt{n}) + n$ (b) Draw the recurrence tree for the recurrence relation: $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + c * n$, Solve the recurrence using recursion tree method.	2 3
2	(a) Discuss one recurrence that can not be solved using master theorem and why ? (b) The recurrence relation $T(n)= 7T(n/2) +n^2$ describe the running time of an algorithm A. Another competing algorithm B has a running time given by the relation $T^1(n)= aT^1(n/4) +n^2$, what is the largest value of **a**, such that algorithm B is asymptotically faster than algorithm A.	2 3
3	What is the largest number of key comparisons made by binary search when searching for a key in the following sorted array? 3, 14, 27, 31, 39, 42, 55, 70, 74, 81, 85, 93, 98 List all the keys in this array that require the largest number of key comparisons during a binary search. Also, find the average number of key comparisons made by binary search for a successful search in this array (assume that each key is searched with equal probability).	5
4	Consider the problem of determining whether a string A[1 … *m*] is a substring of another string B[1 … *n*]; i.e. whether there exist an integer *d* such that A[*i*] = B[*i*+*d*] for all 1<=*i*<=*m*. Give a divide and conquer algorithm to solve the above problem. Also write the recurrence relation for your algorithm and solve that.	5
5	Define matrix chain multiplication problem. Given five matrices A_1 (10 × 20), A_2 (20 × 5), A_3 (5 × 10), A_4 (10 × 5), and A_5 (5 ×20) use dynamic programming to determine the optimal parenthesization order that minimizes the number of scalar multiplications required for matrix chain multiplication. Calculate the minimum number of scalar multiplications needed and provide the optimal parenthesization. Show all the steps clearly.	5
6	Let W={5,10,7,15,12,20,18} and M=35. Using backtracking, find all possible subsets of W that sum to M. Draw a portion of the state space tree that is generated and contain solution.	5
7	In the Job Scheduling problem, the goal is to select the maximum number of mutually compatible (non-overlapping) jobs. Identify the input and output of this problem and state the greedy strategy used to obtain an optimal set of jobs. Using this strategy, determine the maximum number of non-overlapping jobs that can be selected from the following set: Job 1 (1,4), Job 2 (3,5), Job 3 (0,6), Job 4 (5,7), Job 5 (8,9), and Job 6 (5,9).	5

8	A directed graph G is given below, where each vertex is labeled with a unique index. Assume that whenever the traversal has a choice among multiple adjacent vertices, it always selects the vertex with the smallest index first. Perform a DFS (start from vertex 1) on the graph and construct the DFS tree. Classify every edge in the graph as a tree edge, back edge, forward edge, or cross edge. Also, write the parenthesis (discovery–finish) structure generated during the DFS traversal. Finally, determine whether a topological ordering exists for this graph and justify your answer. 	5
9	Consider a symbol set {a, b, c, d, e, f} with corresponding probabilities given as 0.19, 0.23, 0.03, 0.45, 0.05, and 0.05, respectively. Construct the Huffman's tree corresponding to the optimal prefix encoding. Find the average bits needed per character for the optimal prefix code. If you were to encode a text of 10,000 characters using both Huffman encoding and fixed-length encoding, compute the difference in total storage space required.	5
10	Determine the all-pairs shortest paths for a given following graph using Floyd-Warshall's algorithm. Additionally, compare the time complexity of Floyd-Warshall's algorithm with the approach of applying a single-source shortest path algorithm (Bellman-Ford) iteratively for all V vertices in a dense graph to compute the all-pairs shortest paths. 	5